Instituto Superior de Engenharia
Politécnico de Coimbra

Decision Tables

M.Sc. in Informatics Engineering

Slides © 2019 Jorge Barreiros – Parts adapted from original sources

Black-Box Testing

Decision Tables

Overview

- **Decision Tables (DT) model complex, conditional program behavior:**
  - Test cases can be derived from decision tables
  - A DT describes the conditions and actions to perform depending on those conditions.
  - Each column is a rule that associates actions with required conditions.
  - Each rule can originate a test case

Example: Decision table for triangle problem

Condition Stub	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10	R11
C1: a<b+c	F	T	T	T	T	T	T	T	T	T	T
C2: b<a+c	-	F	T	T	T	T	T	T	T	T	T
C3: c<b+a	-	-	F	T	T	T	T	T	T	T	T
C4: a=b	-	-	-	T	T	T	T	F	F	F	F
C5: a=c	-	-	-	T	T	F	F	T	T	F	F
C6: b=c	-	-	-	T	F	T	F	T	F	T	F
A1: not a triangle	x	x	x								
A2: Scalene											x
A3: Isosceles							x		x	x	
A4: Equilateral				x							

Example: Decision table for triangle problem

Condition Stub	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10	R11
C1: a<b+c	F	T	T	T	T	T	T	T	T	T	T
C2: b<a+c	-	F	T	T	T	T	T	T	T	T	T
C3: c<b+a	-	-	F	T	T	T	T	T	T	T	T
C4: a=b	-	-	-	T	T	T	T	F	F	F	F
C5: a=c	-	-	-	T	T	F	F	T	T	F	F
C6: b=c	-	-	-	T	F	T	F	T	F	T	F
A1: not a triangle	x	x	x								
A2: Scalene											x
A3: Isosceles							x		x	x	
A4: Equilateral				x							

impossible

impossible

Example: Decision table for triangle problem

Condition Stub	R1	R2	R3	R4	R7	R9	R10	R11
C1: a<b+c	F	T	T	T	T	T	T	T
C2: b<a+c	-	F	T	T	T	T	T	T
C3: c<b+a	-	-	F	T	T	T	T	T
C4: a=b	-	-	-	T	T	F	F	F
C5: a=c	-	-	-	T	F	T	F	F
C6: b=c	-	-	-	T	F	F	T	F
A1: not a triangle	x	x	x					
A2: Scalene								x
A3: Isosceles					x	x	x	
A4: Equilateral				x				

Example: Decision table for triangle problem

Condition Stub	R1
C1: a<b+c	F
C2: b<a+c	-
C3: c<b+a	-
C4: a=b	-
C5: a=c	-
C6: b=c	-
A1: not a triangle	x
A2: Scalene	
A3: Isosceles	
A4: Equilateral	

Each rule will generate an appropriate test case:

Test Case ID	a	b	c	Output
T1	**4**	**1**	**2**	**Not a triangle**

Example: Decision table for triangle problem

Condition Stub	R2
C1: a<b+c	T
C2: b<a+c	F
C3: c<b+a	-
C4: a=b	-
C5: a=c	-
C6: b=c	-
A1: not a triangle	x
A2: Scalene	
A3: Isosceles	
A4: Equilateral	

Each rule will generate an appropriate test case:

Test Case ID	a	b	c	Output
T1	4	1	2	Not a triangle
T2	2	9	5	Not a triangle

Example: Decision table for triangle problem

Condition Stub	R3
C1: a<b+c	T
C2: b<a+c	T
C3: c<b+a	F
C4: a=b	-
C5: a=c	-
C6: b=c	-
A1: not a triangle	x
A2: Scalene	
A3: Isosceles	
A4: Equilateral	

Each rule will generate an appropriate test case:

Test Case ID	a	b	c	Output
T1	4	1	2	Not a triangle
T2	2	9	6	Not a triangle
T3	3	4	8	Not a triangle

Example: Decision table for triangle problem

Condition Stub	R4
C1: a<b+c	T
C2: b<a+c	T
C3: c<b+a	T
C4: a=b	T
C5: a=c	T
C6: b=c	T
A1: not a triangle	
A2: Scalene	
A3: Isosceles	
A4: Equilateral	x

Each rule will generate an appropriate test case:

Test Case ID	a	b	c	Output
T1	4	1	2	Not a triangle
T2	2	9	6	Not a triangle
T3	3	4	8	Not a triangle
T4	10	10	10	Equilateral

Test Generation

Considering all the rules, the folowing test cases can be derived

Test Case ID	a	b	c	Output
T1	4	1	2	Not a triangle
T2	2	9	6	Not a triangle
T3	3	4	8	Not a triangle
T4	10	10	10	Equilateral
T5	1	9	9	Isosceles
T6	8	2	8	Isosceles
T7	7	7	6	Isosceles
T8	2	3	4	Scalene

Black-Box Testing

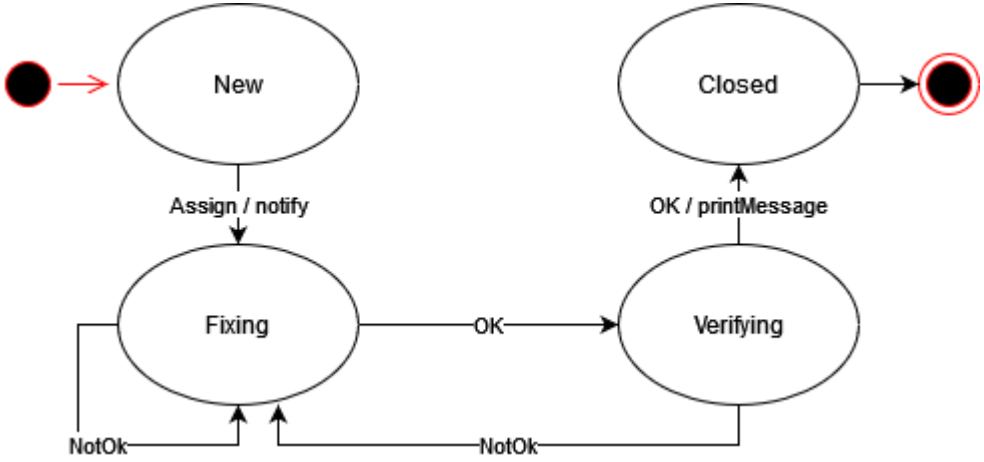
State Transition Testing

Overview

- **State machines are sometimes be used to describe the desired system behavior.**
  - In this case, the behaviour of the system is dependent not only on the applied inputs at a given point in time, but it also depends on the past inputs as well.
  - We can use the information provided by the state machine in order to create appropriate tests (even though we might not have access to the source code)
- **State machines can be represented in different ways.**
  - **Graphical Representation**
  - **State transition table**
  - **Full state transition table**

Graphical State Machine

- Tracking Bug status



Transition Tables

- Full Transition Table
- Reduced Transition Table

State \ Event	Assign	Ok	NotOk
NEW	FIXING		
FIXING		VERIFYING	FIXING
VERIFYING		CLOSED	FIXING
CLOSED			

State	Event	Next State
NEW	Assign	**FIXING**
NEW	Ok	undefined
NEW	NotOk	undefined
FIXING	Assign	undefined
FIXING	Ok	**VERIFYING**
FIXING	NotOk	**FIXING**
VERIFYING	Assign	undefined
VERIFYING	Ok	**CLOSED**
VERIFYING	NotOk	**FIXING**
CLOSED	Assign	undefined
CLOSED	Ok	undefined
CLOSED	NotOk	undefined

N-Switches

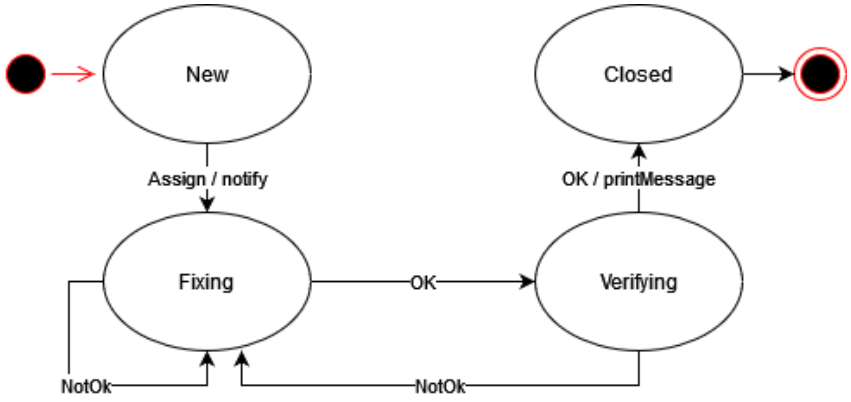
- An *N-Switch* is a sequence of states such that there are exactly N states between the first and last states.
  - **A -> (E1) -> B :** 0-switch – *from state A to state B via edge E1*
  - **A -> (E1) -> B -> (E2) -> C :** 1 –switch
  - **A -> (E1) -> B -> (E2) -> C -> (E3) -> D: 2** –switch
  - **A -> (E1) -> B -> (E2) -> A -> (E3) -> C : 2** –switch
  - **A -> (E1) -> A -> (E2) -> A -> (E1) -> A : 2** –switch

Classical Coverage Criteria

- **These are the "classical" criteria for state transition coverage (ITSQB):**
  - **State Coverage:** All states must be visited at least once in the test suite
  - **Transition Coverage (aka 0-switch coverage):** All transitions must be exercised at least once in the test suite.
  - **N-switch Coverage:** All n-switches must be transversed at least once in the test suite (and also all m-switches for m<n)
  - **Invalid Transition Coverage:** All invalid transitions must be attempted at least once (in individual tests) in the test suite.

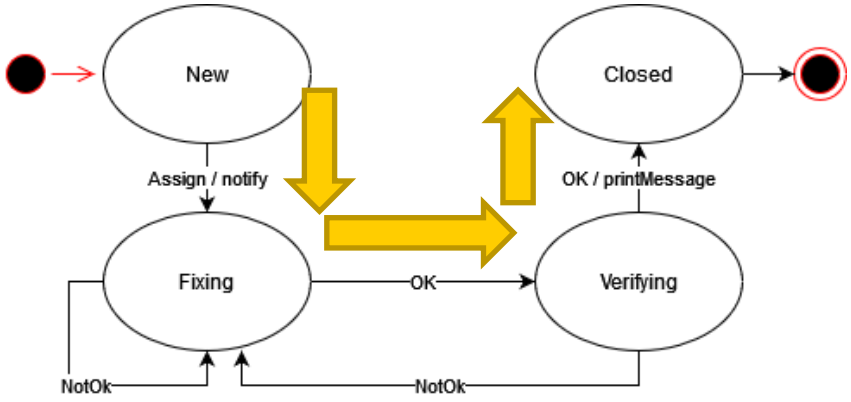
State Coverage

Nr.	States	Inputs
1	New -> Fixing -> Verifying -> Closed	Assign -> OK -> OK



State Coverage

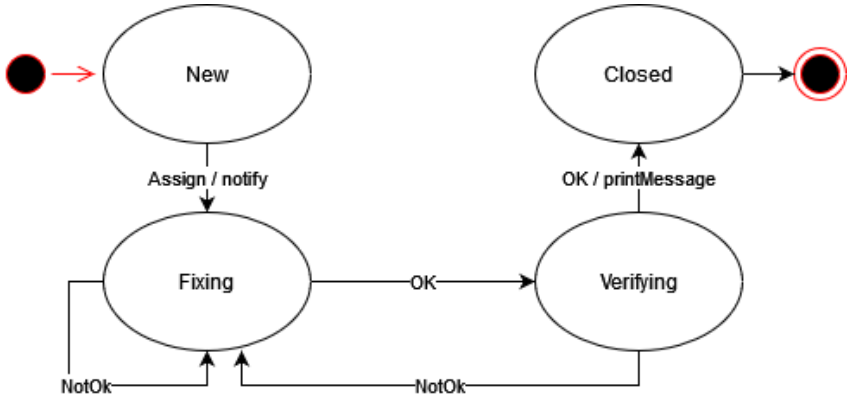
Nr.	States	Inputs
1	New -> Fixing -> Verifying -> Closed	Assign -> OK -> OK



Transition Coverage

- This is one possible solution…

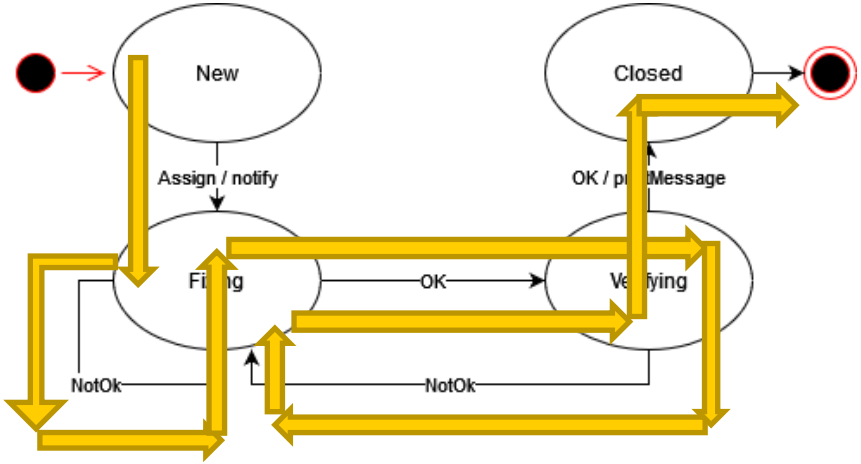
Nr.	Inputs
1	Assign -> OK -> OK
2	Assign -> Not Ok -> OK -> OK
3	Assign -> OK -> NotOk -> OK ->Ok



Transition Coverage

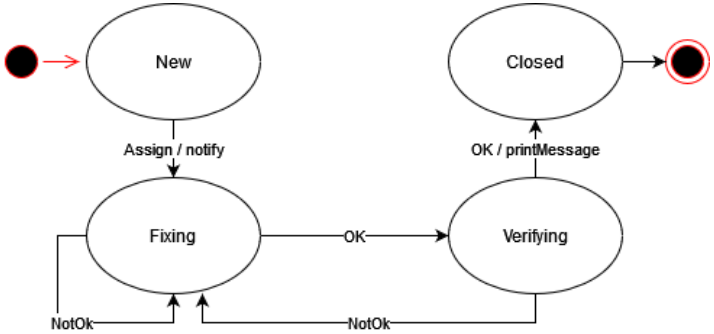
- Though only one test is actually required…

Nr.	Inputs
1	Assign -> NotOk -> OK -> NotOk -> OK -> OK



1-switch Coverage

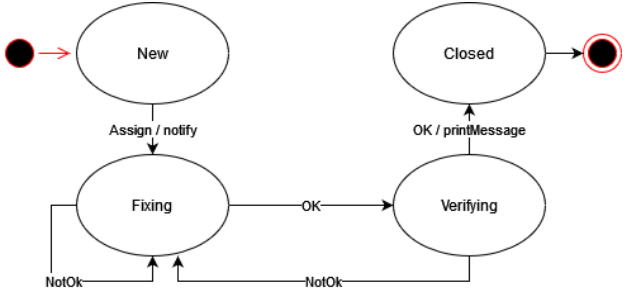
- **Let's systematically identify all 1-switchs.**
  - First write down all the 0-switches and then expand each one with one possible transition.



Nr.	0-switches
1	New -> Fixing
2	Fixing -> Fixing
3	Fixing -> Verifying
4	Verifying ->Fixing
5	Verifying -> Closed

1-switch Coverage

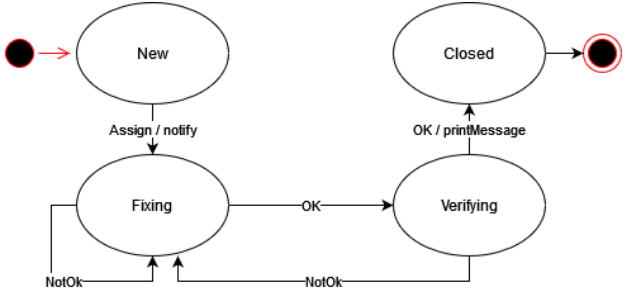
- **Let's systematically identify all 1-switchs.**
  - First write down all the 0-switches and then expand each one with one possible transition.



0-switches	1-switches
New -> Fixing	New -> Fixing -> Fixing
	New -> Fixing -> Verifying
Fixing -> Fixing	Fixing -> Fixing -> Fixing
	Fixing -> Fixing -> Verifying
Fixing -> Verifying	Fixing -> Verifying -> Fixing
	Fixing -> Verifying -> Closed
Verifying ->Fixing	Verifying -> Fixing -> Fixing
	Verifying -> Fixing -> Verifying
Verifying -> Closed	

1-switch Coverage

- **Let's systematically identify all 1-switchs.**
  - First write down all the 0-switches and then expand each one with one possible transition.

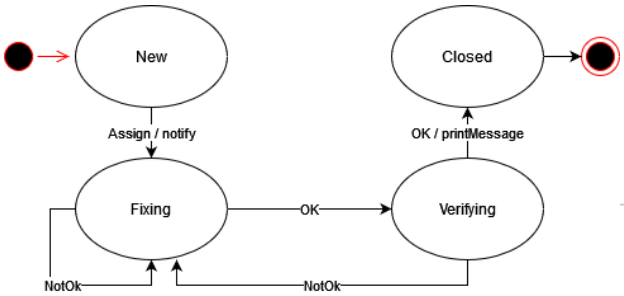


0-switches	1-switches
New -> Fixing	New -> Fixing -> Fixing
	New -> Fixing -> Verifying
Fixing -> Fixing	Fixing -> Fixing -> Fixing
	Fixing -> Fixing -> Verifying
Fixing -> Verifying	Fixing -> Verifying -> Fixing
	Fixing -> Verifying -> Closed
Verifying ->Fixing	Verifying -> Fixing -> Fixing
	Verifying -> Fixing -> Verifying
Verifying -> Closed	

1-switch Coverage

- Building the Test Suite

Nr.	Test Path	Inputs	Satisfied Switches
1	N->F->F->V ->F->V->C	Assign -> NotOK -> OK-> NotOK-> OK ->OK	1, 4, 5, 6, 8

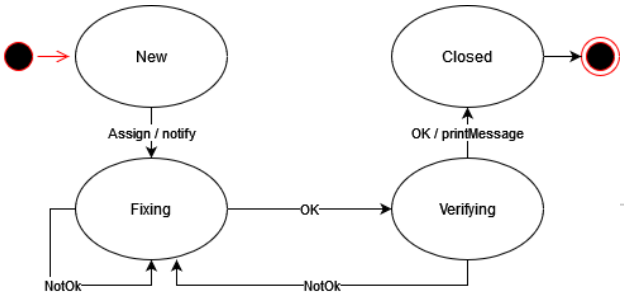


Nr.	1-switches
1	New -> Fixing -> Fixing
2	New -> Fixing -> Verifying
3	Fixing -> Fixing -> Fixing
4	Fixing -> Fixing -> Verifying
5	Fixing -> Verifying -> Fixing
6	Fixing -> Verifying -> Closed
7	Verifying -> Fixing -> Fixing
8	Verifying -> Fixing -> Verifying

1-switch Coverage

- Building the Test Suite

Nr.	Test Path	Inputs	Satisfied Switches
1	N->F->F->V ->F->V->C	Assign -> NotOK -> OK-> NotOK-> OK ->OK	1, 4, 5, 6, 8

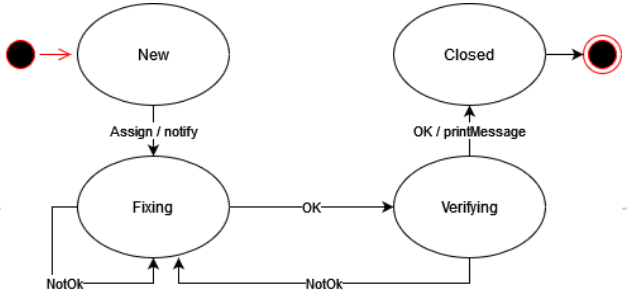


Nr.	1-switches
1	New -> Fixing -> Fixing
2	New -> Fixing -> Verifying
3	Fixing -> Fixing -> Fixing
4	Fixing -> Fixing -> Verifying
5	Fixing -> Verifying -> Fixing
6	Fixing -> Verifying -> Closed
7	Verifying -> Fixing -> Fixing
8	Verifying -> Fixing -> Verifying

1-switch Coverage

- Building the Test Suite

Nr.	Test Path	Inputs	Satisfied Switches
1	N->F->F->V ->F->V->C	Assign -> NotOK -> OK-> NotOK-> OK ->OK	1, 4, 5, 6, 8
2	N-> F ->V -> F -> F-> F-> V ->C	Assign -> Ok -> NotOK -> NotOK -> NotOK -> OK -> OK	2, 3, 4, 5, 6, 7



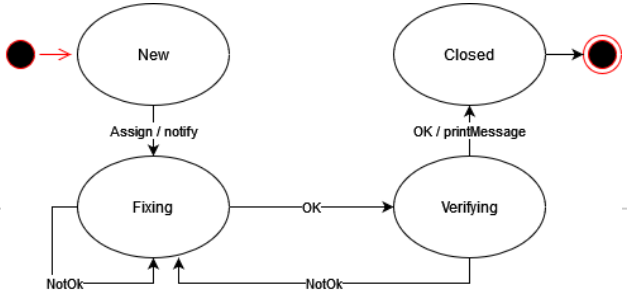
Nr.	1-switches
1	New -> Fixing -> Fixing
2	New -> Fixing -> Verifying
3	Fixing -> Fixing -> Fixing
4	Fixing -> Fixing -> Verifying
5	Fixing -> Verifying -> Fixing
6	Fixing -> Verifying -> Closed
7	Verifying -> Fixing -> Fixing
8	Verifying -> Fixing -> Verifying

1-switch Coverage

- Building the Test Suite

Nr.	Test Path	Inputs	Satisfied Switches
1	N->F->F->V ->F->V->C	Assign -> NotOK -> OK-> NotOK-> OK ->OK	1, 4, 5, 6, 8
2	N-> F ->V -> F -> F-> F-> V ->C	Assign -> Ok -> NotOK -> NotOK -> NotOK -> OK -> OK	2, 3, 4, 5, 6, 7

Is this the minimum number of required tests?

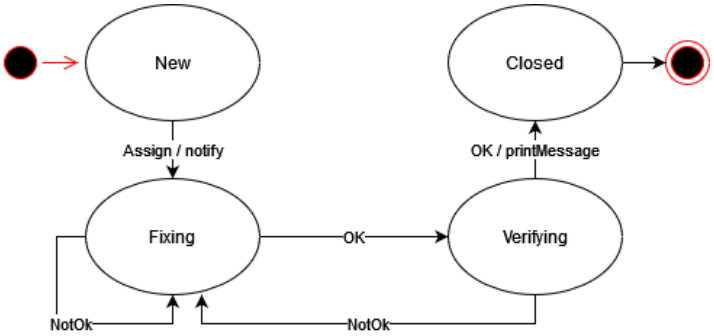


Nr.	1-switches
1	New -> Fixing -> Fixing
2	New -> Fixing -> Verifying
3	Fixing -> Fixing -> Fixing
4	Fixing -> Fixing -> Verifying
5	Fixing -> Verifying -> Fixing
6	Fixing -> Verifying -> Closed
7	Verifying -> Fixing -> Fixing
8	Verifying -> Fixing -> Verifying

Invalid Transition Coverage

- Identify the invalid transitions

State	Valid transitions	Invalid Transitions
New	Assign	OK, NotOk
Fixing	OK, NotOK	Assign
Verifying	OK, NotOK	Assign
Closed		OK, NotOK, Assign

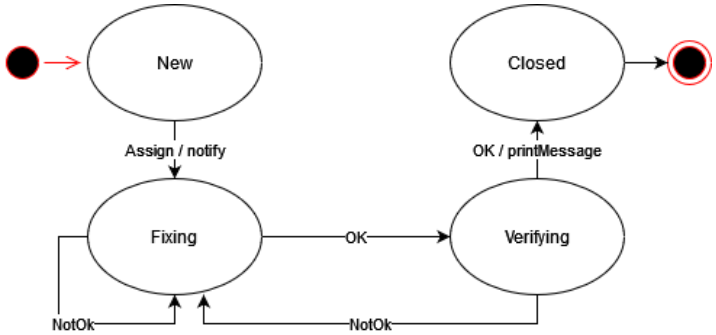


We'll need 7 tests, one per invalid transition

Invalid Transition Coverage

- Create the test suite
  - *Remember – 1 test per invalid transition*

Nr.	Test Path	Inputs
1	N -> ?	OK
2	N -> ?	NotOK
3	N -> F -> ?	Assign -> Assign
4	N -> F -> V -> ?	Assign -> OK -> Assign
5	N -> F -> V -> C -> ?	Assign -> OK -> OK -> OK
6	N -> F -> V -> C -> ?	Assign -> OK -> OK -> NotOK
7	N -> F -> V -> C -> ?	Assign -> OK -> OK -> Assign



State	Valid transitions	Invalid Transitions
New	Assign	OK, NotOk
Fixing	OK, NotOK	Assign
Verifying	OK, NotOK	Assign
Closed		OK, NotOK, Assign